Library build with task selection: does repairing where programs disagree build a better library?

Date: 2026-10-14 onwards (section 2 is updated as results arrive). One RTX 5090. Model: `claude-sonnet-5` for every writer and repair agent. Figure: `figs/` (made by `make_figures.py`). Numbers: `analysis_runs.py`. Code: `scripts/taskselect/library_loop.py`.

Summary

- A skill library is built from nothing by repairing one failing task per round. Two ways to choose the task are compared, with paired repetitions: by **disagreement** between programs (`lex`), and **at random among unsolved tasks** (`unsolved`). Each round writes 27 programs (9 tasks x 3) and runs them on 30 seeds; each arm makes 5 repairs.
- **Noise is large.** Five independent generations from the same empty library have SD 0.027 in mean pass rate, so two single runs differ by SD 0.038 and a gap below 0.075 cannot be shown with two repetitions (0.061 with three, 0.043 with six).
- **So far no evidence that `lex` beats `unsolved`.** With the first 3 programs per task `lex` ended higher in both finished repetitions (+0.032, +0.059). Re-measuring each final library with 3 more programs per task (6 in total) removed the gap (-0.009, +0.002): the lead was writer luck.
- **The repair loop does help.** All four runs end above their round 0 (+0.022 to +0.119). In one run a generic grasp rule was adopted by 23 of 27 programs, and a task nobody repaired improved.
- Repetition 3 is running and repetitions 4 and 5 are queued. A second proposition, validating a repair before its lesson enters the library, is designed but not tested (section 3).

Proposition and hypotheses

Setting. A language model writes a Python program for a robot task from a skill library (markdown files of rules and helpers). When programs fail, an agent repairs one task, and the lesson is written back into the library. The library is the only thing that carries knowledge from one round to the next. Repairing is expensive (about an hour of agent and simulator time per task), so a build repairs only a few tasks. **Which task to repair next decides what the library learns.**

Proposition. The most informative task to repair is the one where the current programs **disagree**: some pass and some fail on the same seeds, many unsolved tasks fail at the same stage, and the failures have varied traces. Disagreement says that the task is neither hopeless nor already solved, and that a fix at a shared failure stage will carry over to other tasks. Repairing by this rule (`lex`) should build a better library than repairing a random unsolved task (`unsolved`) at the same number of repairs.

Hypotheses (all about the library after the same number of repairs, same tasks, same writers):

id	statement	measure	how it can fail
H1 (primary)	the final library of `lex` gives a higher pass rate than that of `unsolved`	mean pass rate over 27 programs (9 tasks x 3), and over 6 programs per task, on 30 seeds	gap within the noise, or negative
H2	`lex` improves a library faster along the rounds	gain over round 0, mean over rounds 1 to 5	no gain difference
H3	`lex` lessons transfer: tasks that were not repaired improve	per-task passes, unrepaired tasks	gains only on repaired tasks
H0 (noise)	one run's mean moves by 0.03 (SD) with no change in the library, so an effect smaller than the detectable gap cannot be shown	five round-0 generations from the empty library	(this is a measured fact, not a claim)

What can be concluded. The smallest gap that two arms can reliably show (5% level, 80% power) is 0.106 with 1 repetition, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6. A result below that is reported as "not shown", not as "no effect". The harness idea (section 3) is a second, separate proposition and is not tested yet.

Method: the library, the loop and the update rule

The library. Markdown files in `.claude/libero/skills/` (`grasp`, `localize`, `manipulation`, `transport`, `README`). Writers read them as context; programs never import them. A build starts from the empty library `L00`, and each repair produces the next one (`L01`, `L02`, ...). Every version is stored in `outputs/<run>/library/Lnn`.

Algorithm.

```
L <- empty library
```

```
repeat 5 times:
    P <- 3 fresh programs per task, written from L          # 9 tasks
    run P on seeds 51..80                                   # 810 trials
    U <- tasks where no program passes every seed
    t <- pick(U)                                            # the only difference between arms
    findings <- repair(t)                                   # up to 5 tries, dev seeds 51..65
    L <- L + lessons(findings)                              # add rules, never remove
final: 3 fresh programs per task from L, run on seeds 51..80

pick(U), arm "unsolved":  a random task of U
pick(U), arm "lex":       the task of U with the largest (split, shares, trace), compared in that order
    split  = how much programs disagree per seed (entropy of how many pass)
    shares = how many tasks of U fail at the same stage
    trace  = how varied the failure traces are
```

A trial's failure stage comes from its API trace only: `crash`, `perceive` (object not found), `no_motion`, `no_close`, `air` (closed on nothing), `slip` (held, then lost), `held_end`, `released` (placed wrong). `LEX.md` has a worked example.

Repair. The agent reads the passing and failing traces of the task's 3 programs, then writes up to 5 task-level programs (no seed branches, no hard-coded coordinates), each run on the 15 dev seeds and diagnosed from its failed trials. The kept program is the one with the most passes (ties: the later one). It writes `findings.md`: root causes, pass counts and "Generalizable Patterns".

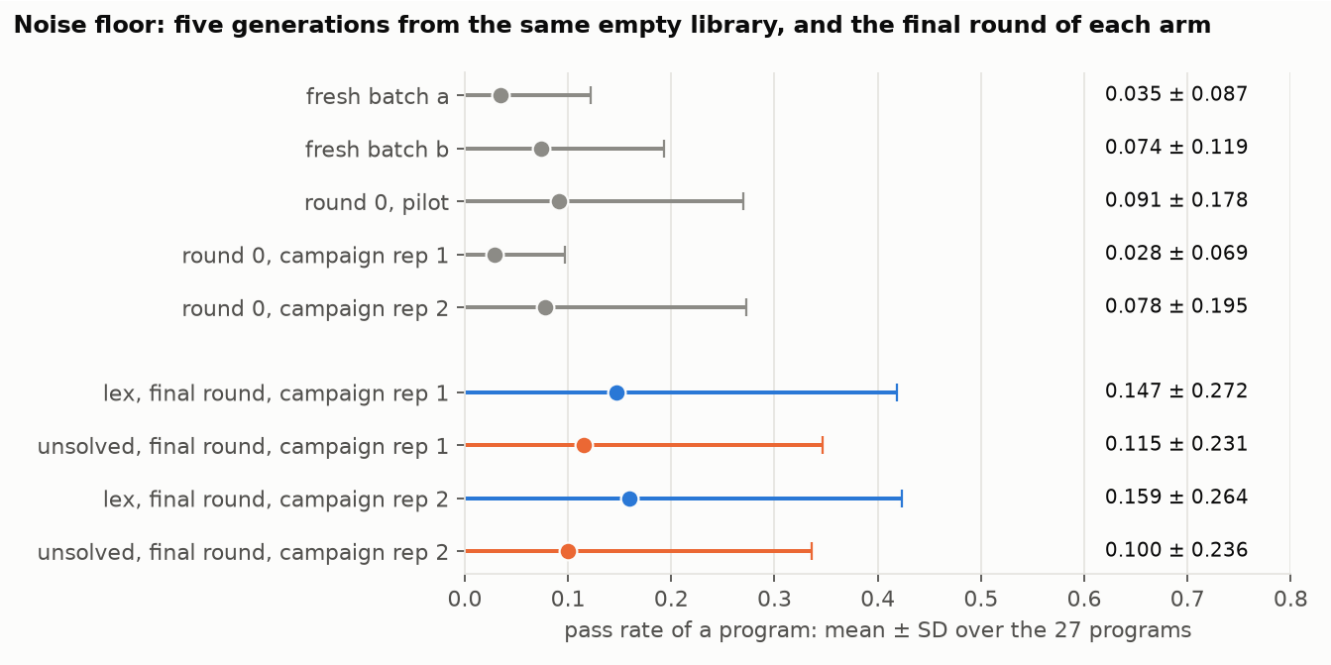
Library update rule (promotion). After the repair, the coordinator reads `findings.md`, edits the skill files and logs it in a ledger (`skill_promotions.jsonl`: files changed, reason). Rules in force:

1. A lesson is added as a short rule with its trigger and evidence, in the file of its category (grasp, localise, place, carry). Generic patterns are preferred over task facts.
2. No rule is ever removed or corrected: the library only grows (4.7 KB after the first repair, 17.6 KB after five in one run).
3. The kept program's pass count on the dev seeds is the only evidence required. There is no test on other seeds, which is the weakness section 3 addresses.
4. Skills may use the allowed APIs only (no simulator state, no `.bddl`/`.xml` reads).

Protocol and controls.

item	value
pool	9 swap tasks: goal (3), spatial (3), object (3); 21 more swap tasks are held out and unused so far
probe seeds	51-80 (30), every round; repair development seeds are 51-65
repetition	round 0 is written once and shared by both arms; arm order alternates; 5 repetitions planned
same in both arms	model (`claude-sonnet-5`), K, writers' prompts, repair budget, seeds, API facts note
differs	only the selection rule
noise control	five independent round-0 generations from the empty library, and a re-measurement of each final library with 3 more programs per task

1. Noise floor



Independent generations of programs from the same empty library, same pool, same seeds (27 programs, 9 tasks x 3, each program run on its seeds). The plot shows the mean and SD over the 27 programs of each program's pass rate, for five generations and for the final round of each arm. Most programs pass nothing and a few pass most seeds, so the SD is large against the mean.

generation	mean pass rate	note
fresh batch a	0.035	15 seeds
fresh batch b	0.074	15 seeds
round 0 of the second pilot repetition	0.091	15 seeds
round 0 of campaign rep 1	0.028	30 seeds
round 0 of campaign rep 2	0.078	30 seeds
mean, SD between generations	**0.062, 0.027**	

- One run's mean moves by 0.027 (SD) with no change to the library. The difference of two single runs has SD 0.038.
- **Smallest gap between two arms that is reliably seen** (5% level, 80% power): 0.106 with 1 repetition per arm, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6.
- **Seeds are not the limit.** With 30 seeds the sampling noise of a run's mean is only 0.005 to 0.008. The noise comes from the programs the writers happen to produce and from the repairs, so the lever is more repetitions or more programs per task.
- Execution adds a little: re-running identical programs on the same seeds changed 11 of 15 per-seed outcomes for one program.

2. Campaign results: 30 seeds, 5 repairs (updated as results arrive)

Last update: 2026-10-18, after repetition 2 (both arms).

Design. Same pool, 3 programs per task, same rules in both arms; every probe uses **30 seeds (51-80)**, each arm makes **5 repairs** (6 probes). The writers' prompts carry the API facts (the point prompt takes one `(x, y)` pair, `import numpy as np`, no Molmo). Five repetitions are planned; in each, round 0 is written once and shared by both arms, and the arm order alternates. Run ids end in `_s30_r<k>`. Driver: `run_reps.py --reps k --seeds 51..80 --rounds 5 --tag _s30`.

Status: reps 1 and 2 done (about 8 to 10 h per arm); rep 3 running; reps 4 and 5 queued.

Mean pass rate over the 27 programs, by round (all seeds of the run).

run	0	1	2	3	4	5
`lex` rep 1	0.028	0.062	0.081	0.048	0.083	0.147
`unsolved` rep 1	0.028	0.063	0.133	0.049	0.094	0.115
`lex` rep 2	0.078	0.152	0.091	0.104	0.072	0.159

run	0	1	2	3	4	5
`unsolved` rep 2	0.078	0.007	0.070	0.177	0.117	0.100

Final round on seeds 66-80 only (no repair used them): `lex` 0.138 and 0.165, `unsolved` 0.121 and 0.094 (reps 1 and 2).

2.1 `lex` against `unsolved`, first 3 programs per task

rep	`lex` final	`unsolved` final	gap	`lex` gain over round 0	`unsolved` gain over round 0	gap in the mean of rounds 1-5
1	0.147	0.115	+0.032	+0.119	+0.086	-0.007
2	0.159	0.100	+0.059	+0.081	+0.022	+0.021
mean	0.153	0.108	+0.045	+0.100	+0.054	+0.007

- `lex` ends higher in both repetitions, also on seeds 66-80, and gains more over its round 0. Averaged over rounds 1 to 5 the two arms are level, so the final-round gap may be luck at the last step.
- The mean gap of +0.045 is 1.7 standard errors (SE of the mean of two pair differences: 0.038 / sqrt(2) = 0.027). Two repetitions detect a gap of 0.075, so this is not a result. Reps 3 to 5 decide.
- Single rounds are very noisy: `unsolved` in rep 2 fell from 0.078 to 0.007 in round 1 (35% crashed trials) and recovered to 0.177 by round 3.
- All four 30-seed runs end above their round 0 (+0.022 to +0.119); only `lex` rep 1 has a 95% interval above 0, so the gain of the repair loop is likely but is not shown by one run.

2.2 Final libraries measured with 6 programs per task

`remeasure.py` writes 3 new programs per task from each final library (read from the run's own library directory) and probes them on the same 30 seeds. With the original 3, every task has 6 programs.

rep	`lex` (6 programs)	`unsolved` (6 programs)	gap	95% interval of the gap
1	0.110 (original 3: 0.147, new 3: 0.074)	0.120 (0.115, 0.125)	-0.009	[-0.151, +0.122]
2	0.104 (0.159, 0.048)	0.101 (0.100, 0.102)	+0.002	[-0.102, +0.090]
mean			-0.003	

- The new programs from the `lex` libraries score far below the original ones (0.074 and 0.048 against 0.147 and 0.159), while the `unsolved` ones match (0.125 and 0.102 against 0.115 and 0.100). The earlier `lex` lead came from the draw of the original programs.
- One process difference: the original programs came from subagents inside the arm's coordinator session, the new ones from a fresh session that read only the library. Both arms are treated alike and `unsolved` shows no shift.
- So with this measure the two arms are level. This is the conclusion to carry forward unless rep 3 and later change it.

2.3 `lex`, repetition 1: what the repairs changed

- **Result.** The mean rose from 0.028 to 0.147 over the five repairs (+0.119, 95% interval [+0.009, +0.247]; bootstrap over tasks and seeds). On seeds 66-80 it rose from 0.025 to 0.138, so this is not repair-seed overfitting. The final value is about 3 noise SDs above fresh round-0 generations.
- **The path is noisy.** The round steps are +0.034, +0.019, -0.033, +0.035, +0.064; their SD (0.036) is about the noise SD. Only the last step is clearly beyond it. The trend is +0.018 per round.
- **Crashed trials** are 0 to 19% (4% in the final round); the first pilot, without the API-facts note, had up to 43%.
- **No task is solved on all 30 seeds** in any round. Final-round passes of 90: salad dressing 51, turn on the stove 30, bowl on the stove 19.

round	task repaired	diagnosis	library addition
0	open the middle drawer	handle count unstable, but handle choice did not decide pass or fail	no motion churn without checking known-good seeds; ordinal targets; never retry from a stale pose
1	bowl on the stove	an absolute GraspNet score threshold rejects every grasp; 4 pick attempts help (5 to 11 of 15)	more retries beat tuning; no absolute score filter
2	bowl on the stove	two placement-target clusters from one stove mask	height-bimodality guard; density-mode centroid
3	bowl between plate and ramekin	"ramekin" scores 0.001, so the betweenness test never ran and the wrong bowl was grasped	read the **settled** gripper width, not the return value of `close_gripper()`; no score filter on landmark objects

round	task repaired	diagnosis	library addition
4	salad dressing	clean trace, "inside footprint", reward 0: release dynamics	one run's pass count cannot validate a change

Only the stove repair of round 1 passed its 15 development seeds with the kept program.

- **One element clearly changed the programs.** The settled-width rule (added after round 3) was used by 0 of 27 programs before it and by **23 of 27 in round 4 and 24 of 27 in round 5**. Programs that use it score higher (0.126 against 0.043), but the 7 programs without it are spread over three tasks, so this shows adoption, not benefit. Best case: salad dressing, 3 passes (round 3) to 42 and 51 (rounds 4 and 5); that task had just been repaired, on the same development seeds.
- **Knowledge crossed tasks.** "Turn on the stove" was never repaired and went 5, 11, 14, 17, 10, 30 of 90, while the stove-bowl and drawer repairs added generic retry and placement rules that appear in its programs.
- `lex` picked the bowl on the stove twice (disagreement stayed high after the first, failed, repair).
- **Library size** went from 4.7 KB (L01) to 17.6 KB (L05), additions only.

What the campaign can claim so far. Five repairs on 30 seeds raised this run's pass rate outside the noise, on seeds no repair used. Not shown: that `lex` beats random selection, or that any single rule caused the gain.

3. A second proposition: validate a repair before its lesson enters the library (not tested yet)

Problem. Evaluation is noisy (the SD between independent generations is 0.027; identical programs scored 7/15 and 13/15 on the same seeds), yet a lesson enters the library on the strength of the kept program's dev-seed pass count (rule 3 above). Luck and wrong diagnoses (an early fix loop went 0/15 on dev and 0/50 held out) therefore become permanent library text, and the library only grows.

Hypotheses.

- H4: a library that only accepts repairs that beat their starting program on fresh seeds scores higher on unseen seeds and on unseen tasks than an ungated one at the same repair budget. It can fail: with a margin of 3 passes of 15 the gate accepts about 16% of useless repairs and only about 55% of a true +20-point repair.
- H5: giving the repair agent a trace diagnosis tool (per-query detection scores, grasped-object position, grasp contrast between passing and failing trials) raises the share of repairs that pass the gate.

Pieces built (committed, untested on the simulator): `harness/why.py` (the diagnosis tool, CPU only), `harness/accept.py` (the gate), `prompts/repair_harness_addendum.md`, and `run_reps.py --treatment harness`.

```
accept(kept, start):   run both on fresh seeds 81..95
    if passes(kept) - passes(start) >= 3:  add the lessons to L
    else:                                   leave L unchanged
```

Planned design: arms `unsolved`, `unsolved + tool`, `unsolved + gate`, `unsolved + both`, at least 4 repetitions, budget counted including the gate's trials, measured on unseen seeds, 6 programs per task, and on the 21 held-out tasks. Section 6 lists the held-out transfer test as the first thing to run after the campaign.

4. Deviations and limits

- **A first pilot is not reported.** Its round 0 had 9 programs patched by hand for API errors (missing `import numpy`, a list passed to the point prompt, a dict passed as a mask), the point-prompt misuse recurred in later rounds (up to 43% crashed trials in one arm), and it used 15 seeds and 3 repairs. It showed the pipeline runs and that the noise is large, and it led to the API-facts note in the writers' prompts. A second pilot repetition (15 seeds, 3 repairs, no patches) is used only for its round-0 generation in the noise floor. All results above come from runs without hand patches.
- **Molmo is off**, as in the upstream TC1 runs; writers are told not to use it.
- **Interrupted runs.** One arm's coordinator session was stopped by mistake and one hit the subscription rate limit for hours; both were restarted from the files on disk with a resume notice. The work done is unchanged.
- **Process difference in the 6-program re-measurement:** the 3 new programs per task come from a fresh session that read only the final library, the original 3 from subagents of the arm's coordinator. Both arms are treated alike.
- Probe seeds 51-65 are also the repair development seeds, so the repaired task is slightly favoured on half of its seeds in both arms; seeds 66-80 are reported separately.

Where things are, by experiment id

All paths are under `aspire/sim/outputs/` (gitignored). The live directory `.claude/libero/skills/` is only a working copy: each run saves its own libraries, so it is restored to the committed version when no run is active.

experiment id	what it is	libraries	probes and selections	repairs
`library_noise_a`, `library_noise_b`	fresh round-0 generations from the empty library	`library/L00`	`round_00/probe.json`	none
`library_lex_r2`, `library_unsolved_r2`	a pilot repetition (15 seeds, 3 repairs; shared round 0)	`library/L00` to `L03`	same	same
`library_lex_s30_r<k>`, `library_unsolved_s30_r<k>`	campaign (k = 1 to 5); `outputs/remeasure/<run>` holds the 3 extra programs per task and their probe (30 seeds 51-80, 5 repairs, 6 probes; shared round 0 per repetition)	`library/L00` to `L05`	same	same
`skill_snapshots/family_start`	skills after the first single-task fix loop; equals the committed `.claude/libero/skills`	5 files	none	none

Each run also has `state.json` (pool, rounds, settings) and `coordinator.md` (the exact prompt it was started with). In each repetition the second arm's round 0 is a byte-identical copy of the shared one (`probe.json` compared with `cmp`).

5. What this supports and what comes next

Supported: the pipeline runs end to end; five repairs raise a library's pass rate beyond the noise in most runs; generic rules spread to programs and to tasks that were not repaired; run-to-run noise is as large as the differences seen.

Not supported: that disagreement selection beats random selection, in either direction.

Next, in order:

1. Finish repetition 3 (and 4, 5 if wanted) and re-measure each final library with 6 programs per task (`remeasure.py`).
2. Evaluate the start library and each arm's final library on the 21 held-out swap tasks.
3. Test the validated-repair proposition (section 3) against `unsolved`.